



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس مبانی رایانش توزیع شده
گزارش تمرین سوم

دانشجویان:

سارا گیتی ۸۱۰۱۰۱۵۰۰

پریا پاسه‌ورز ۸۱۰۱۰۱۳۹۳

فهرست مطالب

۴	فهرست تصاویر
۵	فهرست جداول
۶	۱ سوالات تئوری
۶	۱-۱ چرا داده ها در سیستم های توزیع شده replicate می شوند؟
	۲-۱ تفاوت replication برای افزایش کارایی و replication برای تحمل خطا چیست؟
۷	
۷	۳-۱ مشکل اصلی replication از نظر consistency چیست؟
۸	۴-۱ تفاوت strong consistency و eventual consistency چیست؟
۹	۵-۱ مفهوم monotonic reads چیست؟
۹	۶-۱ مفهوم reads-your-write consistency چیست؟
	۷-۱ چرا در سیستم های توزیع شده نمی توان همزمان همیشه بیشترین -consis-
۱۰	tency و availability و partition tolerance را به صورت کامل داشت؟
۱۱	۲ بخش چهارم: آزمایش و تحلیل
۱۱	۱-۲ شرح بخش
۱۲	۲-۲ سناریو ۱: مشاهده ناسازگاری موقت (Temporary Inconsistency)
۱۳	۳-۲ سناریو ۲: خرابی یک رپلیکا (Replica Failure)
۱۳	۲-۳-۱ بخش الف: مدل Eventual، با خاموش بودن replica3
	۲-۳-۲ بخش ب: مدل Strong، با خاموش بودن یک رپلیکا (Quorum) همچنان
۱۴	قابل دسترس)
۱۵	۲-۳-۳ بخش ج: مدل Strong، با خاموش بودن دو رپلیکا (Quorum غیرقابل دسترس)
۱۶	۴-۲ سناریو ۳: تعارض همزمان (Concurrent Conflict)
۱۷	۵-۲ سناریو ۴: تاثیر تاخیر شبکه (Network Delay)
۱۸	۶-۲ جدول معیارهای کلی

۲۰ ۷-۲ مشکلات و محدودیت‌های پیاده‌سازی

۳ بخش پنجم: توضیحات تکمیلی گزارش نهایی

۲۰ ۱-۳ توضیح معماری سیستم

۲۱ ۲-۳ نحوه اجرای رپلیکاها

۲۲ ۳-۳ نحوه اجرای کلاینت

۲۳ ۴-۳ توضیح مدل Consistency پیاده‌سازی شده

۲۳ ۵-۳ توضیح سیاست Versioning

۲۴ ۶-۳ توضیح سیاست Conflict Resolution

۷-۳ تحلیل تفاوت Strong Consistency و Eventual Consistency بر اساس نتایج

۲۵ آزمایش‌ها

فهرست تصاویر

فهرست جداول

۱	نتایج اندازه‌گیری‌شده‌ی سناریو ۴ به ازای تاخیرهای مختلف	۱۹
۲	جدول معیارهای اندازه‌گیری‌شده در سناریوهای آزمایش	۱۹

۱ سوالات تئوری

۱-۱ چرا داده ها در سیستم های توزیع شده replicate می شوند؟

داده ها در سیستم های توزیع شده Replicate می شوند تا قابلیت اطمینان، دسترسی پذیری و کارایی سیستم افزایش یابد. هنگامی که چندین نسخه از یک داده در گره های مختلف نگهداری شود، خرابی یک گره منجر به از دست رفتن داده یا توقف سرویس نخواهد شد، زیرا نسخه های دیگری از همان داده در سایر گره ها موجود هستند. به این ترتیب تحمل خطای سیستم افزایش می یابد.

همچنین Replication باعث بهبود دسترسی پذیری می شود. کاربران می توانند درخواست های خود را به نزدیک ترین یا در دسترس ترین Replica ارسال کنند و در نتیجه حتی در صورت خرابی برخی گره ها یا لینک های ارتباطی، سیستم همچنان قادر به ارائه سرویس خواهد بود.

یکی دیگر از اهداف Replication افزایش کارایی است. با توزیع درخواست های خواندن بین چندین Replica، بار کاری میان گره ها تقسیم می شود و گلوگاه های عملکردی کاهش می یابد. علاوه بر این، قرار دادن Replica ها در نقاط جغرافیایی مختلف سبب می شود کاربران به نسخه ای نزدیک تر از داده دسترسی پیدا کنند و تأخیر ارتباطی کاهش یابد.

در سیستم های بزرگ و جهانی، Replication همچنین به مقیاس پذیری کمک می کند. با افزایش تعداد کاربران، می توان Replica های بیشتری ایجاد کرد تا حجم درخواست ها بین آنها توزیع شود و سیستم بدون کاهش شدید عملکرد به رشد خود ادامه دهد.

بنابراین، مهم ترین دلایل Replication در سیستم های توزیع شده عبارتند از افزایش قابلیت اطمینان، تحمل خطا، دسترسی پذیری، کارایی، کاهش تأخیر دسترسی و بهبود مقیاس پذیری سیستم.

۲-۱ تفاوت replication برای افزایش کارایی و replication برای تحمل خطا

چیست؟

تکثیر داده برای افزایش کارایی و تکثیر داده برای تحمل خطا اگرچه هر دو بر ایجاد چندین نسخه از داده متکی هستند، اما اهداف متفاوتی را دنبال می‌کنند.

در Replication با هدف افزایش کارایی، نسخه‌های داده در مکان‌های مختلف شبکه و معمولاً نزدیک به کاربران قرار می‌گیرند تا زمان دسترسی به داده کاهش یابد و بار پردازشی بین چندین سرور توزیع شود. در این حالت هدف اصلی کاهش تأخیر، افزایش توان عملیاتی و بهبود مقیاس‌پذیری سیستم است. حتی اگر برخی از Replica ها از دست بروند، تا زمانی که یک نسخه در دسترس باشد، هدف اصلی این نوع Replication همچنان محقق شده است.

در مقابل، Replication با هدف تحمل خطا برای اطمینان از ادامه کار سیستم در صورت خرابی گره‌ها، دیسک‌ها یا لینک‌های ارتباطی انجام می‌شود. در این حالت وجود نسخه‌های متعدد باعث می‌شود که در صورت از کار افتادن یک Replica، نسخه‌های دیگر بتوانند جایگزین آن شوند و از دست رفتن داده یا توقف سرویس رخ ندهد. بنابراین هدف اصلی حفظ قابلیت اطمینان و دسترس‌پذیری سیستم است، نه لزوماً افزایش سرعت دسترسی.

به عبارت دیگر، در Replication برای کارایی تمرکز بر نزدیک کردن داده به کاربران و توزیع بار است، در حالی که در Replication برای تحمل خطا تمرکز بر حفظ سرویس و جلوگیری از از دست رفتن داده در هنگام خرابی اجزای سیستم است. هرچند در بسیاری از سیستم‌های توزیع‌شده مدرن، یک مکانیزم Replication به طور همزمان هر دو هدف را تا حدی تأمین می‌کند.

۳-۱ مشکل اصلی replication از نظر consistency چیست؟

مشکل اصلی Replication از نظر Consistency این است که پس از ایجاد چندین نسخه از یک داده، باید اطمینان حاصل شود که همه این نسخه‌ها وضعیت یکسانی را مشاهده می‌کنند. هنگامی که یک عملیات نوشتن روی یکی از Replica ها انجام می‌شود، تغییرات باید به سایر Replica ها نیز منتقل شوند. از آنجا که این انتقال زمان‌بر است و در یک سیستم توزیع‌شده تأخیر شبکه، خرابی گره‌ها و از دست رفتن پیام‌ها وجود دارد، ممکن است برای مدتی نسخه‌های

مختلف داده دارای مقادیر متفاوتی باشند.

در نتیجه، کاربران مختلف ممکن است در یک زمان واحد مقادیر متفاوتی از یک داده را مشاهده کنند. به عنوان مثال، یک کاربر ممکن است داده به‌روزشده را از یک Replica دریافت کند، در حالی که کاربر دیگر هنوز همان داده را از Replica دیگری به صورت قدیمی مشاهده کند. این وضعیت باعث ناسازگاری میان نسخه‌ها می‌شود.

بنابراین چالش اساسی Replication این است که هرچه تعداد Replicas بیشتر و پراکندگی جغرافیایی آن‌ها گسترده‌تر باشد، هماهنگ نگه داشتن همه نسخه‌ها دشوارتر و پرهزینه‌تر می‌شود. حفظ سازگاری قوی نیازمند تبادل پیام و هماهنگی بیشتر میان گره‌ها است که موجب افزایش تأخیر و کاهش کارایی می‌شود. به همین دلیل در سیستم‌های توزیع‌شده معمولاً میان Availability، Consistency و Performance نوعی موازنه و مصالحه وجود دارد.

۴-۱ تفاوت strong consistency و eventual consistency چیست؟

در مدل Strong Consistency، پس از انجام موفق یک عملیات نوشتن، تمام عملیات خواندن بعدی در هر نقطه از سیستم همواره جدیدترین مقدار داده را مشاهده می‌کنند. به عبارت دیگر، سیستم طوری رفتار می‌کند که گویی تنها یک نسخه از داده وجود دارد و همه کاربران همیشه آخرین به‌روزرسانی را می‌بینند. برای دستیابی به این سطح از سازگاری، Replicas باید پیش از پاسخ‌گویی به درخواست‌ها با یکدیگر هماهنگ شوند که این امر هزینه ارتباطی و تأخیر را افزایش می‌دهد.

در مقابل، در مدل Eventual Consistency تضمینی وجود ندارد که بلافاصله پس از یک عملیات نوشتن، همه Replicas ها مقدار جدید را مشاهده کنند. ممکن است برای مدتی برخی نسخه‌ها داده قدیمی و برخی نسخه‌ها داده جدید را نگهداری کنند. با این حال، اگر هیچ به‌روزرسانی جدیدی انجام نشود، سیستم در نهایت به وضعیتی می‌رسد که همه Replicas ها یکسان شده و آخرین مقدار داده را در اختیار خواهند داشت.

بنابراین تفاوت اصلی این دو مدل در زمان تضمین سازگاری است. در Strong Consistency سازگاری به صورت فوری تضمین می‌شود و همه کاربران همواره آخرین مقدار را مشاهده می‌کنند،

در حالی که در Eventual Consistency سازگاری با تأخیر حاصل می‌شود و برای مدتی امکان مشاهده مقادیر متفاوت توسط کاربران مختلف وجود دارد. Strong Consistency سازگاری بیشتری فراهم می‌کند اما هزینه عملکردی بالاتری دارد، در حالی که Eventual Consistency کارایی، مقیاس‌پذیری و دسترس‌پذیری بیشتری را با پذیرش ناسازگاری موقت فراهم می‌کند.

۵-۱ مفهوم monotonic reads چیست؟

Monotonic Reads یکی از مدل‌های سازگاری متمرکز بر کلاینت است که تضمین می‌کند اگر یک کاربر یک مقدار خاص از داده را مشاهده کرده باشد، در خواندن‌های بعدی هرگز نسخه‌ای قدیمی‌تر از آن داده را نخواهد دید.

به عبارت دیگر، پس از اینکه یک کلاینت نتیجه یک به‌روزرسانی را مشاهده کرد، تمام عملیات خواندن بعدی او باید حداقل همان نسخه یا نسخه‌های جدیدتر داده را برگردانند. این مدل مانع از آن می‌شود که کاربر در اثر دسترسی به Replica های مختلف، به عقب بازگردد و داده‌ای قدیمی‌تر از آنچه قبلاً دیده است مشاهده کند.

برای مثال فرض کنید مقدار یک رکورد ابتدا ۱۰ باشد و سپس به ۲۰ تغییر کند. اگر یک کاربر در یک خواندن مقدار ۲۰ را مشاهده کند، در مدل Monotonic Reads تضمین می‌شود که در خواندن‌های بعدی هرگز مقدار ۱۰ را نبیند، حتی اگر درخواست او به Replica دیگری ارسال شود. او ممکن است مقدار ۲۰ یا مقدار جدیدتری مانند ۳۰ را مشاهده کند، اما نه مقدار قدیمی‌تر ۱۰. این مدل به ویژه در سیستم‌های مبتنی بر Replication اهمیت دارد، زیرا به دلیل تأخیر در انتشار به‌روزرسانی‌ها، Replica های مختلف ممکن است در یک لحظه دارای نسخه‌های متفاوتی از داده باشند. Monotonic Reads تجربه‌ای سازگارتر برای کاربر فراهم می‌کند و از مشاهده داده‌های قدیمی پس از مشاهده داده‌های جدید جلوگیری می‌نماید.

۶-۱ مفهوم reads-your-write consistency چیست؟

Reads-Your-Writes Consistency یکی از مدل‌های سازگاری متمرکز بر کلاینت است که تضمین می‌کند اگر یک کاربر یا فرایند یک عملیات نوشتن را با موفقیت انجام دهد، در تمام

عملیات‌های خواندن بعدی خود حداقل همان مقدار نوشته‌شده یا نسخه‌های جدیدتر را مشاهده خواهد کرد.

به عبارت دیگر، پس از آنکه یک کاربر داده‌ای را به‌روزرسانی کرد، هرگز نباید در خواندن‌های بعدی خود نسخه‌ای قدیمی‌تر از داده را ببیند. این ویژگی از بروز وضعیتی جلوگیری می‌کند که در آن کاربر تغییرات خود را اعمال کرده باشد اما به دلیل تأخیر در همگام‌سازی Replica ها، هنگام خواندن مجدد داده هنوز نسخه قدیمی را مشاهده کند.

برای مثال، فرض کنید کاربری وضعیت پروفایل خود را از «Offline» به «Online» تغییر می‌دهد. در صورت وجود Reads-Your-Writes Consistency، این کاربر در خواندن‌های بعدی حتماً وضعیت «Online» یا نسخه‌های جدیدتر را مشاهده خواهد کرد و هرگز وضعیت قدیمی «Offline» را نخواهد دید، حتی اگر درخواست‌های او به Replica های مختلف ارسال شوند. این مدل در سیستم‌های دارای Replication اهمیت زیادی دارد، زیرا تأخیر در انتشار به‌روزرسانی‌ها می‌تواند باعث شود برخی Replica ها هنوز داده قدیمی را نگهداری کنند. Reads-Your-Writes Consistency تجربه‌ای طبیعی و قابل انتظار برای کاربران فراهم می‌کند، زیرا هر کاربر بلافاصله اثر تغییرات خود را مشاهده می‌کند، حتی اگر سایر کاربران هنوز این تغییرات را نبینند.

۷-۱ چرا در سیستم های توزیع شده نمی توان همزمان همیشه بیشترین

consistency و availability و partition tolerance را به صورت کامل

داشت؟

طبق قضیه CAP، در یک سیستم توزیع‌شده نمی‌توان به طور همزمان و در شرایط وجود Partition شبکه، هر سه ویژگی Consistency، Availability و ePartition Toleranc0 را به صورت کامل تضمین کرد.

Consistency به این معناست که همه کاربران در هر لحظه آخرین مقدار داده را مشاهده کنند. Availability بیان می‌کند که هر درخواست دریافتی باید پاسخ دریافت کند، حتی اگر بخشی از سیستم دچار مشکل شده باشد. Partition Tolerance نیز به این معناست که سیستم بتواند با وجود قطع ارتباط یا از دست رفتن پیام‌ها بین بخش‌های مختلف شبکه به کار خود ادامه دهد.

زمانی که یک Partition در شبکه رخ می‌دهد، برخی گره‌ها دیگر قادر به برقراری ارتباط با یکدیگر نیستند. در این شرایط اگر بخواهیم Consistency را حفظ کنیم، باید برخی درخواست‌ها را تا زمان برقراری مجدد ارتباط متوقف یا رد کنیم تا از ناسازگاری داده‌ها جلوگیری شود. در نتیجه Availability کاهش می‌یابد. از طرف دیگر، اگر بخواهیم Availability را حفظ کنیم، باید به گره‌ها اجازه دهیم حتی بدون هماهنگی با سایر گره‌ها به درخواست‌ها پاسخ دهند. در این صورت ممکن است کاربران مختلف مقادیر متفاوتی از داده را مشاهده کنند و Consistency نقض شود.

از آنجا که در سیستم‌های توزیع‌شده واقعی وقوع Partition اجتناب‌ناپذیر است، معمولاً Partition Tolerance یک الزام محسوب می‌شود. بنابراین هنگام بروز Partition، سیستم ناچار است بین Consistency و Availability یکی را در اولویت قرار دهد و نمی‌تواند هر دو را به طور کامل تضمین کند.

در نتیجه، دلیل اصلی این محدودیت آن است که هنگام قطع ارتباط بین بخش‌های مختلف سیستم، حفظ سازگاری کامل نیازمند محدود کردن پاسخ‌دهی است و حفظ پاسخ‌دهی کامل مستلزم پذیرش ناسازگاری موقت داده‌ها است. به همین دلیل دستیابی همزمان به بالاترین سطح Consistency، Availability و Partition Tolerance در یک سیستم توزیع‌شده امکان‌پذیر نیست.

۲ بخش چهارم: آزمایش و تحلیل

۱-۲ شرح بخش

در این بخش، سیستم Key-Value Store توزیع‌شده پیاده‌سازیشده در سه نسخه مجزا (replica1، replica2، replica3) با استفاده از Docker Compose اجرا شده و چهار سناریوی آزمایشی مشخص‌شده در تمرین روی آن انجام گرفته است.

۲-۲ سناریو ۱: مشاهده ناسازگاری موقت (Temporary Inconsistency)

این سناریو با هدف بررسی رفتار سیستم در مدل Eventual Consistency اجرا شده است. ابتدا مقدار $x=10$ روی replica1 نوشته شده، سپس بلافاصله از replica2 خوانده شده است.

مثال ۱.۲. خروجی ترمینال اجرای سناریو ۱:

```

1 $ curl -s -X POST http://localhost:8080/put -d
   '{"key":"x","value":"10"}'
2 PUT replica1 -> HTTP 200 [OK] round_trip=3.2ms
3   body: {"data": {"key":"x","value":"10","version":1,
4           "updated_by":"replica1"}, "latency_ms": 0.593}
5
6 $ curl -s 'http://localhost:8081/get?key=x'
7 GET(immediate) replica2 -> HTTP 200 [OK] round_trip=4.3ms
8   body: {"data": {"key":"x","value":"10","version":1,
9           "updated_by":"replica1"}, "latency_ms": 0.032}

```

در این اجرای خاص، تاخیر شبکه میان کانتینرها (داخل بستر Docker bridge network) به اندازه‌ی کم بود که فرآیند تکثیر ناهمگام (Async Replication) پیش از ارسال درخواست GET بعدی به اتمام رسید؛ بنابراین خواندن بلافاصله از replica2 مقدار قدیمی یا خطای not found را بازنگرداند و همان مقدار $x=10$ با $version=1$ برگردانده شد. تنها یک بار نظرسنجی (poll) برای تایید همگرایی لازم بود و زمان همگرایی اندازه‌گیری شده برابر با ۵/۵۸ میلی‌ثانیه بود؛ یعنی هیچ خواندن ناسازگاری (Stale Read) در این اجرا مشاهده نشد.

تحلیل: این نتیجه نشان میدهد که اگرچه مدل Eventual Consistency تضمینی برای خواندن فوری مقدار جدید ندارد، اما در محیط‌هایی با تاخیر شبکه‌ی بسیار پایین (مانند شبکه‌ی داخلی Docker روی یک ماشین واحد) ممکن است در عمل پنجره‌ی ناسازگاری آنقدر کوچک باشد که در بیشتر اجراها قابل مشاهده نباشد. این موضوع دقیقاً همان رفتاری است که در سناریوی ۴ (تأثیر تاخیر شبکه) به صورت کنترل‌شده بازتولید و اندازه‌گیری شده است؛ جایی که با تزریق

تاخیر مصنوعی، پنجره‌ی ناسازگاری به‌وضوح قابل مشاهده می‌شود.

۳-۲ سناریو ۲: خرابی یک رپلیکا (Replica Failure)

این سناریو در سه بخش، رفتار سیستم را در هر دو مدل Strong Consistency و Eventual هنگام از کار افتادن یک یا چند رپلیکا بررسی می‌کند.

۱-۳-۲ بخش الف: مدل Eventual، با خاموش بودن replica3

مثال ۲.۲. خروجی ترمینال بخش الف:

```
1 $ docker stop replica3
2 replica3
3
4 $ curl -s -X POST http://localhost:8080/put -d
   '{"key": "y", "value": "42"}'
5 PUT(replica3 down, eventual) replica1 -> HTTP 200 [OK]
   round_trip=2.8ms
6   body: {"data": {"key": "y", "value": "42", "version": 1,
7           "updated_by": "replica1"}, "latency_ms": 0.225}
8
9 $ docker start replica3
10 replica3
11
12 $ curl -s 'http://localhost:8082/get?key=y'
13 GET(replica3 after recovery) replica3 -> HTTP 404 [FAIL]
14   body: {"success": false, "error": "key not found"}
```

با وجود خاموش بودن replica3، عملیات PUT روی replica1 با موفقیت کامل و در زمان بسیار

کوتاه (۲/۸ میلیثانیه) انجام شد، زیرا در مدل Eventual Consistency نوشتن محلی به تنهایی برای موفقیت‌آمیز بودن عملیات کافی است و تلاش برای تکثیر به سایر رپلیکاها به صورت ناهمگام و در پس‌زمینه (Goroutine) انجام می‌شود. پس از راه‌اندازی مجدد replica3، بررسی مقدار y روی آن نشان داد که این رپلیکا هنوز مقدار جدید را دریافت نکرده است (خطای key not found - 404).

تحلیل: این نتیجه یک محدودیت مهم در پیاده‌سازی فعلی را آشکار میکند: پیام تکثیر ارسالی از replica1 به replica3 در لحظه‌ی ارسال (که replica3 خاموش بود) با خطای اتصال مواجه و کنار گذاشته شد؛ این پیام پس از روشن شدن مجدد replica3 دوباره ارسال نمیشود، چرا که سازوکار Retry یا صف پیام (Message Queue) برای تکثیرهای ناموفق در سیستم وجود ندارد. در نتیجه، replica3 تا انجام نشدن یک عملیات نوشتن جدید روی کلید y، همچنان مقدار قدیمی (یا عدم وجود مقدار) را نگه میدارد. این رفتار با توضیحات بخش «مشکلات و محدودیت‌ها» سازگار است.

۲-۳-۲ بخش ب: مدل Strong، با خاموش بودن یک رپلیکا (Quorum همچنان قابل دسترس)

مثال ۳.۲. خروجی ترمینال بخش ب:

```
1 $ docker stop replica3
2 replica3
3
4 $ curl -s -X POST http://localhost:8080/put -d
   '{"key": "y", "value": "99"}'
5 PUT(strong, 1 peer down) replica1 -> HTTP 200 [OK]
   round_trip=9.6ms
6 body: {"data": {"key": "y", "value": "99", "version": 1,
7         "updated_by": "replica1"}, "latency_ms": 6.073}
```

با خاموش بودن تنها یکی از دو رپلیکای هم‌تا (replica3)، عملیات PUT در مدل Strong

Consistency همچنان با موفقیت انجام شد، زیرا حد نصاب اکثریت (Quorum) برای سه رپلیکا برابر با ۲ است و این عدد با تایید replica1 (محلی) به علاوه‌ی تایید replica2 تامین میشود. زمان پاسخ این عملیات (۹/۶ میلی‌ثانیه، با تاخیر سرور ۶/۰۷۳ میلیثانیه) محسوسا بیشتر از حالت مشابه در مدل Eventual است، زیرا کلاینت تا دریافت تایید همتای دوم منتظر می‌ماند.

۲-۳-۳ بخش ج: مدل Strong، با خاموش بودن دو رپلیکا (Quorum غیرقاب‌لدسترس)

مثال ۴.۲. خروجی ترمینال بخش ج:

```
1 $ docker stop replica2
2 replica2
3
4 $ curl -s -X POST http://localhost:8080/put -d
    '{"key": "y", "value": "100"}'
5 PUT(strong, 2 peers down) replica1 -> HTTP 503 [FAIL]
    round_trip=5003.5ms
6 body: {"success": false, "error": "quorum not reached:
7       quorum not reached: got 0/1 acks, errors: [
8       replica2: context deadline exceeded,
9       replica3: context deadline exceeded]"}
```

با خاموش شدن replica2 علاوه بر replica3، فقط replica1 (نود محلی) در دسترس باقی ماند که برای تامین حد نصاب اکثریت (۲ از ۳) کافی نیست. عملیات PUT پس از سپری شدن زمان Timeout پیکربندی‌شده (۵ ثانیه برای هر تلاش تکتیر) با خطای HTTP 503 و پیام صریح quorum not reached رد شد.

تحلیل کلی سناریو ۲: مقایسه‌ی سه بخش فوق به‌روشنی تفاوت بنیادین دو مدل سازگاری را در برابر خرابی نشان میدهد. مدل Eventual همواره در صورت در دسترس بودن نود محلی پاسخ موفق میدهد (هزینه‌ی آن، احتمال ناسازگاری موقت سایر رپلیکاهاست)، در حالی که مدل

Strong برای تضمین سازگاری، در ازای از دست دادن دسترسی (Availability) هنگام نبود اکثریت، حاضر به رد کردن عملیات می‌شود؛ نمونه‌ی عملی از قضیه‌ی CAP.

۲-۴ سناریو ۳: تعارض همزمان (Concurrent Conflict)

در این سناریو، دو درخواست PUT با مقادیر متفاوت برای کلید یکسان z، به صورت همزمان (با استفاده از دو Thread مجزا در اسکریپت آزمایش) به replica1 و replica2 ارسال شدند.

مثال ۵.۲. خروجی ترمینال سناریو ۳:

```

1 $ curl -s -X POST http://localhost:8080/put -d
   '{"key":"z","value":"from_r1"}' &
2 $ curl -s -X POST http://localhost:8081/put -d
   '{"key":"z","value":"from_r2"}' &
3
4 PUT(concurrent, r1) replica1 -> HTTP 200 [OK]
   round_trip=2.7ms
5 body: {"data": {"value":"from_r1","version":1,
6         "updated_by":"replica1"}}
7 PUT(concurrent, r2) replica2 -> HTTP 200 [OK]
   round_trip=3.4ms
8 body: {"data": {"value":"from_r2","version":1,
9         "updated_by":"replica2"}}
10
11 ) 2 (
12 GET(final, replica1) -> value: "from_r2", version: 1,
   updated_by: "replica2"
13 GET(final, replica2) -> value: "from_r2", version: 1,
   updated_by: "replica2"
```



```
14 GET(final, replica3) -> value: "from_r2", version: 1,
    updated_by: "replica2"
```

هر دو نوشتن، نسخه‌ی (version) برابر با ۱ ایجاد کردند، زیرا هر دو رپلیکا پیش از دریافت پیام تکثیر طرف مقابل، کلید z را خالی می‌دیدند؛ این دقیقاً تعریف یک تعارض واقعی (True Conflict) است، نه یک خواندن ناسازگار موقت. پس از تبادل پیام‌های تکثیر میان رپلیکاها، هر سه رپلیکا به‌طور مستقل به یک مقدار نهایی واحد یعنی "from_r2" همگرا شدند.

تحلیل: سیاست تفکیک تعارض پیاده‌سازی شده، Last-Write-Wins بر اساس شناسه‌ی رپلیکا است: در صورت برابر بودن نسخه و متفاوت بودن مقدار، رپلیکایی که شناسه‌ی آن از نظر حروف الفبا بزرگتر است برنده می‌شود. از آنجا که "replica1" > "replica2"، مقدار ارسالی از replica2 انتخاب شد. این قانون به‌صورت کاملاً قطعی (Deterministic) است، یعنی هر رپلیکایی که هر دو نسخه‌ی متعارض را دریافت کند، مستقل از سایرین به همین نتیجه می‌رسد؛ به همین دلیل همگرایی نهایی هر سه رپلیکا به یک مقدار واحد، بدون نیاز به هماهنگ‌کننده‌ی مرکزی، تضمین شده است.

۲-۵ سناریو ۴: تاثیر تاخیر شبکه (Network Delay)

در این سناریو، یک تاخیر مصنوعی (network_delay_ms) پیش از ارسال هر پیام تکثیر از replica1 به همتایان خود، با سه مقدار ۵۰۰ و ۲۰۰۰ میلی‌ثانیه اعمال و آزمایش تکرار شد. برای هر مقدار، زمان پاسخ عملیات PUT و زمان همگرایی replica2 (با نظرسنجی هر ۱۰۰ میلی‌ثانیه) اندازه‌گیری شد.

مثال ۶.۲. نمونه‌ی از خروجی نظرسنجی برای تاخیر ۵۰۰ میلی‌ثانیه (نمایش خواندن‌های ناسازگار پیش از همگرایی):

```
1 GET(poll 1, delay=500ms) replica2 -> HTTP 404 [FAIL]
2 GET(poll 2, delay=500ms) replica2 -> HTTP 404 [FAIL]
3 GET(poll 3, delay=500ms) replica2 -> HTTP 404 [FAIL]
```

```

4 GET(poll 4, delay=500ms) replica2 -> HTTP 404 [FAIL]
5 GET(poll 5, delay=500ms) replica2 -> HTTP 404 [FAIL]
6 GET(poll 6, delay=500ms) replica2 -> HTTP 200 [OK]
7   body: {"data": {"value": "A", "version": 1}}

```

تحلیل: همانگونه که جدول ۱ نشان میدهد، زمان پاسخ PUT تقریباً مستقل از مقدار تأخیر تزریق‌شده و در هر سه حالت در حدود $1/3$ تا $1/8$ میلی‌ثانیه باقی ماند. این رفتار مستقیماً ناشی از معماری مدل Eventual Consistency است: کلاینت بلافاصله پس از نوشتن محلی پاسخ دریافت میکند و هرگز منتظر تکمیل عملیات تکثیر (که در پس‌زمینه و با تأخیر اجرا میشود) نمی‌ماند.

در مقابل، زمان همگرایی به‌صورت تقریباً خطی با مقدار تأخیر تزریق‌شده رشد کرد (≈ 4)، $551 \approx$ و $2044 \approx$ میلی‌ثانیه به ازای تأخیرهای ۵ و ۱۹ بار خواندن ناسازگار (Stale Read به‌صورت خطای 404) از replica2 پنجره‌ی ناسازگاری مستقیماً تابعی از سرعت کانال تکثیر است. در آزمایش با تأخیر ۵۰۰ و ۲۰۰۰ میلی‌ثانیه، به ترتیب ۵ و ۱۹ بار خواندن ناسازگار (Stale Read به‌صورت خطای 404) از replica2 پیش از همگرایی نهایی مشاهده شد؛ این نتیجه به‌وضوح مبادله‌ی بنیادین (Trade-off) میان تأخیر نوشتن کم و طول پنجره‌ی ناسازگاری در مدل Eventual Consistency را اثبات میکند.

۲-۶ جدول معیارهای کلی

جدول ۲ خلاصه‌ی از معیارهای کلیدی اندازه‌گیری‌شده در تمامی سناریوها را، مطابق با ساختار خواسته‌شده در تمرین، نمایش میدهد.

* این سناریوها برای اندازه‌گیری «زمان همگرایی» یا «خواندن ناسازگار» طراحی نشده‌اند؛ تمرکز آنها بر بررسی موفقیت/شکست عملیات PUT است.

جدول ۱: نتایج اندازه‌گیری‌شده‌ی سناریو ۴ به ازای تاخیرهای مختلف

تاخیر تزریق‌شده (ms)	زمان پاسخ PUT (ms)	زمان همگرایی (ms)	تعداد نظرسنجی
۰	۱/۸۰	۴/۰۱	۱
۵۰۰	۱/۵۰	۵۵۱/۳۸	۶
۲۰۰۰	۱/۳۵	۲۰۴۴/۲۵	۲۰

جدول ۲: جدول معیارهای اندازه‌گیری‌شده در سناریوهای آزمایش

مدل	سناریو	زمان PUT (ms)	زمان همگرایی (ms)	خواندن ناسازگار
Eventual	۱ - ناسازگاری موقت	۳/۱۷	۵/۵۸	۰
Eventual	۲الف - خرابی (۱ خاموش)	۲/۷۸	ندارد*	ندارد*
Strong	۲ب - خرابی (۱ خاموش، Quorum برقرار)	۹/۶۰	۰ (همزمان)	۰
Strong	۲ج - خرابی (۲ خاموش، Quorum ناممکن)	ناموفق (503)	ناموفق	ناموفق
Eventual	۳ - تعارض همزمان	۳/۳۵ / ۲/۷۰	≈ ۲۰۰۰	ندارد*
Eventual	۴ - تاخیر ms	۱/۸۰	۴/۰۱	۰
Eventual	۴ - تاخیر ms	۱/۵۰	۵۵۱/۳۸	۵
Eventual	۴ - تاخیر ms	۱/۳۵	۲۰۴۴/۲۵	۱۹

۷-۲ مشکلات و محدودیت‌های پیاده‌سازی

با وجود اجرای موفق چهار سناریوی آزمایشی، پیاده‌سازی فعلی دارای محدودیت‌های زیر است:

۱. عدم تلاش مجدد در تکثیر ناهمگام (No Retry on Async Replication): همانطور که در سناریوی ۲ مشاهده شد، در صورتی که یک پیام تکثیر به دلیل خاموش بودن رپلیکای مقصد ارسال نشود، هیچ سازوکاری برای ارسال مجدد آن پس از بازگشت رپلیکا به مدار وجود ندارد؛ رپلیکای بازیابی‌شده تا انجام نشدن یک عملیات نوشتن جدید روی همان کلید، مقدار قدیمی یا خالی را حفظ میکند.

۲. نبود Two-Phase Commit واقعی در مدل Strong: در پیاده‌سازی فعلی، نوشتن محلی پیش از تایید رسیدن به حد نصاب اکثریت انجام میشود. در صورت شکست Quorum، رپلیکای محلی همچنان مقدار جدید (تاییدنشده) را نگه میدارد؛ این رفتار ساده‌سازی‌شده نسبت به پروتکل‌های واقعی مانند Two-Phase Commit یا Raft است.

۳. عدم وجود هماهنگ‌کننده‌ی واحد برای خواندن در مدل Strong: عملیات GET در هر دو مدل، صرفاً از رپلیکای دریافت‌کننده‌ی درخواست خوانده میشود؛ در مدل Strong این رفتار با فرض اینکه نوشتن‌های تاییدشده حتماً به اکثریت رپلیکاها رسیده‌اند قابل قبول است، اما یک هماهنگ‌کننده‌ی Primary رسمی برای تضمین قطعی این موضوع پیاده‌سازی نشده است.

۳ بخش پنجم: توضیحات تکمیلی گزارش نهایی

۱-۳ توضیح معماری سیستم

سیستم پیاده‌سازی‌شده از سه سرویس مستقل replica1، replica2، replica3 تشکیل شده است که هر کدام به صورت یک Process جدا و در یک Container مجزای Docker اجرا می‌شوند. هر رپلیکا حافظه داخلی مستقل خود (In-Memory Store) را نگه می‌دارد و هیچ متغیر مشترکی

میان رپلیکاها وجود ندارد؛ تمام ارتباطات صرفاً از طریق درخواست‌های HTTP روی شبکه انجام می‌شود.

کلاینت (که می‌تواند یک برنامه خط‌فرمان یا اسکریپت آزمایش باشد) درخواست‌های PUT و GET را به یکی از رپلیکاها ارسال می‌کند. هنگامی که یک رپلیکا درخواست PUT دریافت می‌کند، ابتدا مقدار را در حافظه محلی خود ذخیره کرده و سپس بسته به مدل سازگاری پیکربندی‌شده، آن مقدار را به دو رپلیکای دیگر از طریق نقطه پایانی داخلی `/internal/replicate` ارسال می‌کند. سه رپلیکا روی یک شبکه Docker Bridge مشترک با نام `kv-net` قرار دارند که در آن، نام هر Container به‌صورت خودکار به آدرس IP داخلی همان رپلیکا تبدیل می‌شود (مثلاً `replica2:8081`).

نکته مهم در این معماری، تفکیک دو نوع ارتباط است: ارتباط کلاینت با رپلیکاها از طریق پورت‌های منتقل‌شده به `localhost` ماشین میزبان انجام می‌شود (مثلاً `http://localhost:8080`)، در حالی که ارتباط میان خود رپلیکاها از طریق نام Container در شبکه داخلی Docker برقرار می‌شود. این تفکیک باعث می‌شود ارتباط بین رپلیکاها کاملاً روی شبکه واقعی (نه حافظه مشترک) انجام شود.

۲-۳ نحوه اجرای رپلیکاها

برای اجرای کامل سیستم با سه رپلیکا، از Docker Compose استفاده شده است. دستور زیر هر سه سرویس را به‌صورت همزمان Build و اجرا می‌کند:

مثال ۱.۳. دستور راه‌اندازی رپلیکاها:

```
1 cd HW3
2 docker compose up --build -d
```

پس از اجرای این دستور، می‌توان سلامت هر رپلیکا را با درخواست به نقطه پایانی `/health` بررسی کرد:

مثال ۲.۳. بررسی سلامت رپلیکاها:

```
1 curl -s http://localhost:8080/health
2 curl -s http://localhost:8081/health
3 curl -s http://localhost:8082/health
```

هر رپلیکا با یک فایل پیکربندی JSON جداگانه (configs/replicaN.json) اجرا می‌شود که شامل شناسه رپلیکا، شماره پورت، مدل سازگاری، فهرست رپلیکاها و مقدار تاخیر مصنوعی است. برای تغییر مدل سازگاری یا تزریق تاخیر، کافی است این فایل‌ها ویرایش و سرویس مربوطه با دستور `docker compose restart` مجدداً راه‌اندازی شود.

۳-۳ نحوه اجرای کلاینت

علاوه بر اسکریپت خودکار آزمایش، یک کلاینت تعاملی خط فرمان نیز پیاده‌سازی شده است که امکان ارسال دستی درخواست‌های PUT و GET را فراهم می‌کند. اجرای آن به صورت زیر است:

مثال ۳.۳. اجرای کلاینت تعاملی:

```
1 cd client
2 go run . -addr http://localhost:8080
```

پس از اجرا، کلاینت دستورات زیر را می‌پذیرد:

مثال ۴.۳. نمونه دستورات کلاینت:

```
1 PUT x 10
2 GET x
3 HEALTH
4 SWITCH http://localhost:8081
5 EXIT
```

دستور SWITCH امکان تغییر رپلیکای مقصد را بدون نیاز به اجرای مجدد برنامه فراهم می‌کند، که برای مشاهده دستی پدیده ناسازگاری موقت (مانند سناریو ۱) بسیار مفید است.

۳-۴ توضیح مدل Consistency پیاده‌سازی‌شده

در این پروژه دو مدل سازگاری به صورت کامل پیاده‌سازی شده‌اند که با تنظیم فیلد `consistency_mode` در فایل پیکربندی هر رپلیکا قابل انتخاب هستند.

در مدل `Eventual`، هنگام دریافت یک درخواست `PUT`، رپلیکا ابتدا مقدار را در حافظه محلی خود می‌نویسد و بلافاصله پاسخ موفقیت را به کلاینت برمی‌گرداند. سپس، به صورت مستقل و در پس‌زمینه (با استفاده از `Goroutine`)، تلاش می‌کند مقدار جدید را به سایر رپلیکاها نیز ارسال کند. اگر این ارسال با خطا مواجه شود، خطا فقط در گزارش ثبت می‌شود و تاثیری بر پاسخ کلاینت ندارد.

در مدل `Strong`، پس از نوشتن محلی، رپلیکا منتظر می‌ماند تا حداقل تعداد لازم برای تشکیل حد نصاب اکثریت (`Quorum`)، دریافت موفق پیام تکثیر را تایید کنند. برای سه رپلیکا، این حد نصاب برابر با ۲ است (یک رپلیکای محلی به علاوه یک همتا). در صورتی که این تعداد تایید در بازه زمانی مشخص (`Timeout`) به دست نیاید، عملیات با خطای `HTTP 503` و پیام `quorum not reached` رد می‌شود.

عملیات `GET` در هر دو مدل، همیشه از حافظه محلی همان رپلیکایی خوانده می‌شود که درخواست را دریافت کرده است؛ هیچ هماهنگ‌سازی اضافی در زمان خواندن انجام نمی‌شود.

۳-۵ توضیح سیاست Versioning

هر مقدار ذخیره‌شده در سیستم به همراه یک شماره نسخه (`Version`) نگهداری می‌شود. ساختار هر رکورد به این صورت است:

مثال ۵.۳. ساختار یک رکورد ذخیره‌شده:

```
1 {  
2   "key": "x",
```

```
3   "value": "10",  
4   "version": 3,  
5   "updated_by": "replica1",  
6   "updated_at": "2026-01-01T12:00:00Z"  
7 }
```

هر بار که یک کلاینت روی یک رپلیکا عملیات PUT انجام می‌دهد، آن رپلیکا شماره نسخه را نسبت به مقدار قبلی همان کلید یک واحد افزایش می‌دهد (و اگر کلید قبلاً وجود نداشته باشد، نسخه برابر با ۱ قرار می‌گیرد). فیلد updated_by نیز همواره شناسه رپلیکایی است که نوشتن اصلی روی آن انجام شده است.

هنگامی که یک رپلیکا یک پیام تکثیر دریافت می‌کند، نسخه دریافتی را با نسخه محلی کلید مقایسه می‌کند. اگر نسخه دریافتی بزرگتر باشد، مقدار جدید جایگزین مقدار قبلی می‌شود. اگر نسخه دریافتی کوچکتر باشد، مقدار دریافتی نادیده گرفته می‌شود تا از بازنویسی یک مقدار جدیدتر با مقدار قدیمی‌تر جلوگیری شود. این سیاست تضمین می‌کند که هیچ‌گاه یک نسخه قدیمی‌تر جایگزین یک نسخه جدیدتر نشود.

۳-۶ توضیح سیاست Conflict Resolution

در شرایطی که دو رپلیکا به صورت تقریباً همزمان روی یک کلید یکسان نوشته شوند (مانند سناریو ۳)، ممکن است هر دو نسخه با شماره نسخه برابر اما مقدار متفاوت ایجاد شوند. این وضعیت یک تعارض واقعی (True Conflict) محسوب می‌شود.

برای حل این تعارض، سیاست Last-Write-Wins بر اساس شناسه رپلیکا پیاده‌سازی شده است: در صورتی که دو رکورد نسخه برابر اما مقدار متفاوت داشته باشند، رکوردی که فیلد updated_by آن از نظر حروف‌الفبا (Lexicographic) بزرگتر باشد، به عنوان برنده انتخاب می‌شود. به عنوان مثال، چون "replica1" > "replica2"، در صورت تعارض میان این دو رپلیکا، همواره مقدار نوشته‌شده توسط replica2 برنده خواهد بود.

مهم‌ترین ویژگی این سیاست، قطعی (Deterministic) بودن آن است؛ یعنی هر رپلیکایی که هر دو نسخه متعارض را دریافت کند، بدون نیاز به هماهنگی با سایر گره‌ها، مستقلاً به همان نتیجه می‌رسد. این ویژگی تضمین می‌کند که در نهایت همه رپلیکاها بدون نیاز به یک هماهنگ‌کننده مرکزی به یک مقدار واحد همگرا شوند.

۳-۷ تحلیل تفاوت Strong Consistency و Eventual Consistency بر اساس نتایج آزمایش‌ها

نتایج چهار سناریوی آزمایش‌شده، تفاوت عملی این دو مدل را به خوبی نشان می‌دهد. از نظر زمان پاسخ نوشتن، مدل Eventual همواره سریع‌تر است زیرا کلاینت بلافاصله پس از نوشتن محلی پاسخ دریافت می‌کند (در سناریو ۴، زمان پاسخ PUT مستقل از مقدار تأخیر توزیع‌شده و در حدود ۱ تا ۲ میلی‌ثانیه باقی ماند). در مقابل، مدل Strong به دلیل نیاز به دریافت تایید، زمان پاسخ بیشتری دارد (در سناریو ۲، زمان پاسخ به ۹.۶ میلی‌ثانیه رسید). از نظر دسترس‌پذیری (Availability) در برابر خرابی، مدل Eventual حتی با خاموش بودن همه رپلیکاها نیز همچنان به درخواست‌های نوشتن پاسخ موفق می‌دهد (سناریو ۲الف)، در حالی که مدل Strong در صورت نبود حد نصاب اکثریت، عملیات را به طور کامل رد می‌کند (سناریو ۲ج).

از نظر سازگاری داده، مدل Eventual امکان مشاهده مقادیر قدیمی برای مدت کوتاهی پس از نوشتن را دارد (سناریو ۱ و سناریو ۴)، در حالی که مدل Strong پس از موفقیت یک عملیات نوشتن، تضمین می‌دهد که حداقل اکثریت رپلیکاها به مقدار جدید رسیده‌اند.